DevSync

رواد مصر الرقمية

GRADUATION PROJECT

Project Planning & Management.

DevSync: Secure AI Project Collaborator

Flutter graduation project | Planning phase: 2 Oct - 16 Oct 2026

TEAM OF 5

Donia Alhosin

Bishay Talaat

Yousef Mohy

Emad Hany

Saga Maher

REPORT DURATION

2 Oct 2026 - 16 Oct 2026

2 weeks

Report map

This report covers the five required components of Project Planning & Management. Compared with the first version, it uses a rotating full-stack ownership model, a corrected task board split, a task dependency board, an install-link release path and a future roadmap.

Required component	Where to find it
Project Proposal: overview, objectives and scope	Part 1, sections 1 to 4
Project Plan: Gantt chart, milestones, deliverables, resource allocation	Part 2, sections 5 to 8
Task Assignment and Roles	Part 3, sections 9 to 12
Risk Assessment and Mitigation Plan	Part 4, sections 13 and 14
KPIs (Key Performance Indicators)	Part 5, sections 15 and 16
Future work and ideas to strengthen the project	Part 6, section 17

PART 1

Project Proposal

1. Project overview

DevSync is a Flutter mobile app where developers find collaborators for open-source projects. AI matches each user with projects that fit their skills, using their GitHub activity and skill set. The matching prompts are evaluated and optimized with user feedback.

Problem. Developers who want to contribute to open source struggle to find projects that fit them, and maintainers struggle to find the right contributors.

Solution. An app that reads a developer's GitHub activity, scores how well they fit each project, and lets them send and receive collaboration requests with real-time alerts.

Item	Detail
Project	DevSync: Secure AI Project Collaborator
Type	Flutter graduation project, group of 5
Timeline	Implementation 16 Oct 2026 to 30 Nov 2026: six 1-week sprints plus a 4-day final window
Backend	Firebase (Auth, Firestore, Cloud Messaging) with Cloud Functions for notification triggers and AI calls
Platform	Android, Free and Pro flavors
Distribution	Not published on an app store. A signed, obfuscated build is shared through an install link (GitHub Releases or Firebase App Distribution). An Android App Bundle is also produced to meet the course requirement.
Task tracking	Jira Scrum board

Item	Detail
Version control	Git and GitHub with branches and pull requests

2. Objectives

- Build a working app with AI-driven matching between developers and projects.
- Secure sign-in with Google and email using Firebase UI Auth.
- Real-time alerts for collaboration requests and project updates with Firebase Cloud Messaging.
- Smooth, modern animations with implicit and advanced Flutter animation widgets.
- Ship Free and Pro flavors as obfuscated release builds, shared through an install link.
- Give every team member hands-on experience in every layer of the app (UI, logic, Firestore, tests).
- Deliver full technical documentation, a README, clean Git history and a security-reviewed app.

3. Scope

With a fixed deadline of 30 Nov, scope is split into must-have and if-time-allows items. Must-have items are protected; the rest are cut first.

Must have	If time allows
Sign-in with Google and email Profile with GitHub data and skills Project listing (seed data and user-added projects) AI match feed with score and reason Like and skip feedback on matches Collaboration requests (send, accept, decline) FCM alerts for requests and updates Core animations Free and Pro flavors Obfuscated release builds and install link Documentation and README	Task management for accepted collaborations (shared board, tasks, task alerts) Prompt-quality dashboard Extra advanced animation sequences Extra Pro-only features

Out of scope: iOS release, app store publishing, in-app payments (Pro is a flavor configuration only), hosting of project code (DevSync links to GitHub). A web platform is planned as a later phase (see section 17).

4. Technical approach

Area	Decision
Framework	Flutter and Dart
Backend	Firebase: Auth, Firestore, Cloud Messaging and Cloud Functions. No separate server.
Authentication	Firebase UI Auth with Google and email providers
Database	Cloud Firestore
Notifications	Firebase Cloud Messaging (FCM), triggered from Cloud Functions

Area	Decision
AI matching	LLM prompts that score user-to-project fit, with prompt evaluation and optimization. Calls go through a Cloud Function so the API key never sits in the app.
Animations	Implicit animations, TweenAnimationBuilder and transition widgets
Flavors	free and pro, each with its own app ID, icon and Firebase configuration
Release	Code obfuscation, signed build for the install link, plus Android App Bundle (.aab)
Version control	Git and GitHub
Task tracking	Jira (Scrum board, sprints, GitHub integration)

AI matching and prompt engineering

- Collect a user's GitHub activity (languages, repositories, contributions) and declared skills.
- Build a structured profile and send it to the model with a matching prompt.
- The model returns a fit score and a short reason for each project.
- Test prompt versions on a fixed set of sample profiles and projects.
- Compare results with user feedback (like, skip) and keep the best prompt version.

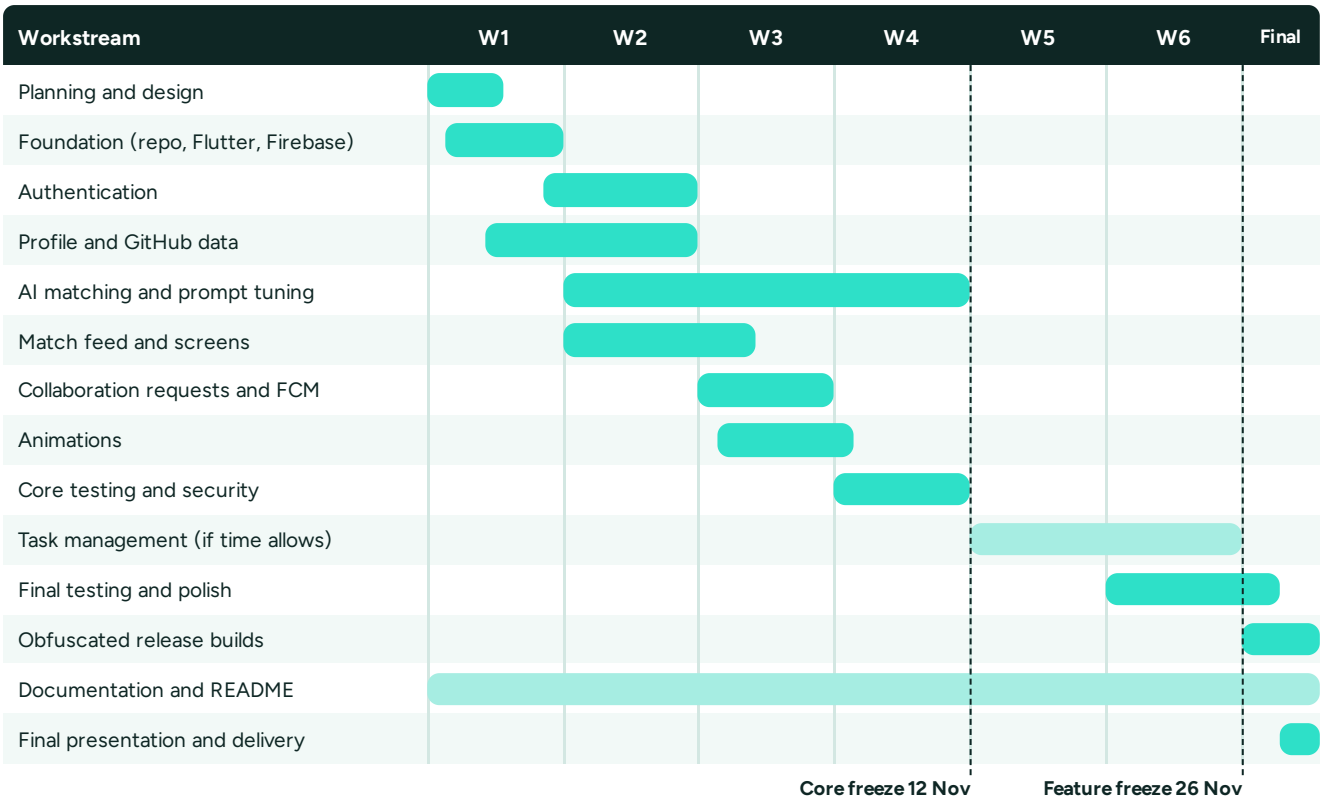
Branding

Merge logo (two paths join into one node). Palette: Teal #2EE0C8, Forest #0E2624, Mint #E8F5F2, Surface #153532. Fonts: Bricolage Grotesque for headings and Figtree for body text.

PART 2

Project Plan

5. Timeline, Gantt chart and milestones



Light bars are optional or continuous work. Dashed lines mark the core freeze (12 Nov) and the feature freeze (26 Nov).

Period	Milestone (end of period)
W1: 16-22 Oct	Foundation ready: repo and branches, Flutter app with flavors and theme, Firebase project and data model, wireframes approved
W2: 23-29 Oct	Users can sign in, see a profile and a first AI-ranked match feed, and like or skip a match
W3: 30 Oct-5 Nov	Collaboration requests and FCM alerts work end to end, animations in, prompts evaluated
W4: 6-12 Nov	Core complete: all must-have features tested, security review done, first obfuscated build tested. Core freeze.
W5-W6: 13-26 Nov	Task management board for accepted collaborations (if time allows), polish and full testing. If the core is behind, this time is used as buffer. Feature freeze 26 Nov.
27-30 Nov	Final obfuscated Free and Pro builds, install link, documentation, README and final presentation delivered on 30 Nov

6. Deliverables

Everyone contributes to every deliverable. The **Lead** is accountable for it being finished and reviewed.

Deliverable	Lead	Due
Project planning report (this document)	Donia	Done
Brand guide and branding presentation	Donia	Done
Core app: all must-have features, tested	All	12 Nov
Test and security report	Saga	12 Nov (final update 30 Nov)
Task management board (if time allows)	Donia	26 Nov
Flutter source code on GitHub with branching history and pull requests	All	Continuous, final 30 Nov
Obfuscated builds (Free and Pro), App Bundle and install link	Yousef	30 Nov
Technical documentation and README (each person documents their own slices)	Saga	30 Nov
Final presentation and demo	Bishay	30 Nov

7. Resource allocation

People. Five team members at about 12 hours per week each, roughly 80 hours per member and 400 team hours over the project. This is a planning estimate; adjust it to your real schedule.

Resource	Used for	Notes
Flutter and Dart SDK, Android Studio or VS Code	App development	Free
Firebase (Auth, Firestore, FCM, Functions)	Backend, sign-in, notifications, AI calls	Free plan to start. Cloud Functions needs the Blaze (pay-as-you-go) plan, which has a free quota. Confirm before Week 3.
GitHub (code and Releases)	Source control, pull requests, reviews, install link	Free
Firebase App Distribution (optional)	Sharing test builds with testers	Free. Alternative to GitHub Releases.
Jira	Scrum board, sprints, task tracking	Free plan
LLM API	AI matching and prompt evaluation	Usage-capped. Budget to be confirmed.
Test devices	Manual testing	At least two real Android devices and one emulator
Local bundle test	Android App Bundle check	Local bundle testing is enough for the deadline

8. Management approach

- **Method:** Agile Scrum with six 1-week sprints and a 4-day final window.
- **Meetings:** sprint planning at the start of each week, a 10-minute daily stand-up, a sprint review and short retrospective at the end of each week.
- **Tracking:** Jira Scrum board (To do, In progress, In review, Done), connected to GitHub. Every task has an owner, a reviewer and a Jira key (for example DS-12).
- **Definition of done:** merged through a reviewed pull request, tested by someone other than the author, and documented.
- **Feature freeze:** core features on 12 Nov, task management on 26 Nov. 27-30 Nov is for release builds, documentation and delivery only.

Git and GitHub strategy

Item	Rule
Branches	main (stable), develop (integration), feature/name, fix/name, release/version
Pull requests	Every change goes through a pull request with at least one reviewer. No direct pushes to main.
Commits	Short, clear messages that start with the Jira key
Jira link	Jira key in branch names and commits, for example feature/DS-12-match-feed and DS-12: add match score meter
Releases	Tagged on main after each weekly milestone

Communication and documentation

What	How	When
Daily questions	Team chat	Ongoing
Stand-up	Short call	Daily, 10 minutes
Sprint review	Call with demo	End of each week
Code review	GitHub pull requests	Each change
Supervisor update	Progress summary by the project manager	Weekly

- **README:** description, features, setup steps, flavors, build commands, folder structure, contribution guide, install link.
- **Technical documentation:** architecture, data model, AI matching and prompt design, authentication, notifications, security, testing and release process.
- Documentation is updated in the same pull request as the feature it describes.

PART 3

Task Assignment and Roles

9. Working model: rotating full-stack slices

Every team member works on every layer of the app. Work is divided into **feature slices** (screens, logic, Firestore and tests for one feature). Each slice has one owner and one reviewer, and ownership rotates every sprint so knowledge spreads.

- One owner builds a slice end to end; a different person reviews the pull request.
- Quality and security are shared habits: before merge, someone other than the author tests the feature.
- Pair on the first day of each new slice to reduce ramp-up time.
- UI never waits for backend: agree the data shape in the PR description, build against mock data, then swap in the real data.

Coordination hats

Hats are small coordination duties on top of the slices. They do not limit who can work on an area.

Member	Coordination hat	Duties on top of feature work
Donia Alhosin	Project manager	Planning, Jira board and sprints, supervisor updates, brand guide, Firestore data model owner
Bishay Talaat	Design lead	Design system, animation consistency, final presentation
Yousef Mohy	App architecture and release lead	Project structure, state management, flavors, release builds and install link
Emad Hany	AI and prompt lead	Matching prompts, evaluation set, prompt versions, AI documentation
Saga Maher	Quality and security lead	CI checks, test and security checklists, evaluation harness, regression runs

10. Slice plan by period

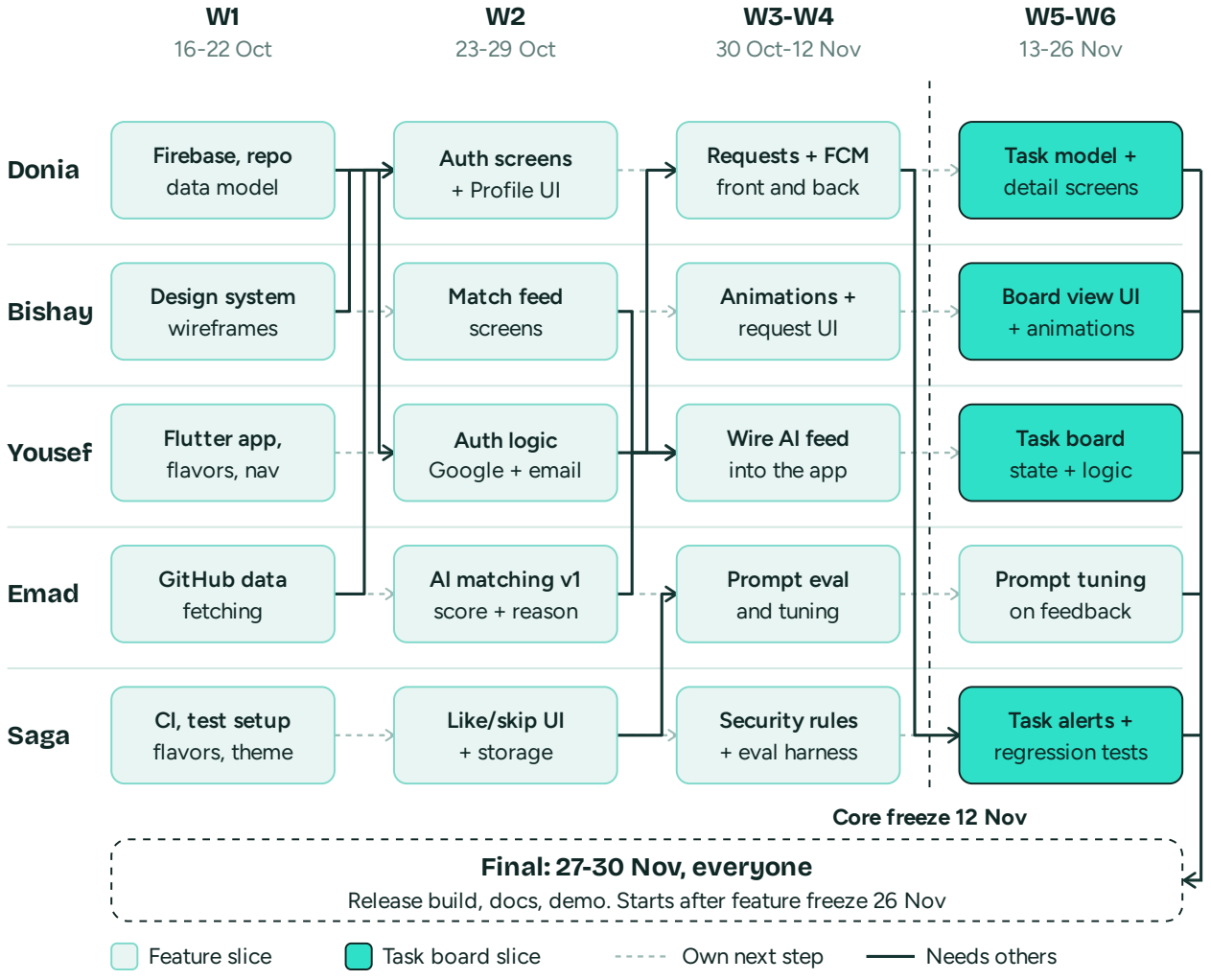
Period	Donia	Bishay	Yousef	Emad	Saga
W1 (pairs)	Repo, Firebase, data model (with Yousef)	Design system, wireframes	Flutter project, flavors, navigation (with Donia)	GitHub data fetching	CI, test setup, flavors and theme (with Bishay)
W2	Welcome, home, auth, profile + Firestore	Match feed screens	Auth logic	AI matching v1	Like/skip feedback: UI and storage
W3-W4	Collaboration requests and FCM, front and back	Animations and request screens	Wire AI results into the app (with Emad)	Prompt evaluation and tuning	Security rules, permissions, first obfuscated build, eval harness

Period	Donia	Bishay	Yousef	Emad	Saga
W5-W6	Task model, rules and detail screens	Board view UI and animations	Task board state and logic	Prompt tuning on real feedback	Task alerts and regression tests
Final	Everyone: release build, docs, demo, split by section				

Task board split (fixed). The task board is split by what each person builds so nobody overlaps: Donia owns the data model, rules and detail/create screens; Yousef owns state and logic; Bishay owns the board view and animations; Saga owns task alerts, reusing Donia's FCM pattern.

11. Task dependency board

Solid arrows show a task that needs another person's output first. Dashed arrows show the same person continuing to the next slice.



Tasks in the same column that depend on each other are listed in the handoff table below.

Handoff table: who finishes before who starts

Task	Who waits on it	"Done" means
Firebase, repo, data model (Donia)	Everyone	Collections and field names merged to develop and written in the README
Design system (Bishay)	Profile, feed	Theme and shared widgets merged
Flutter app and navigation (Yousef)	All screens	App runs with routes stubbed
GitHub data (Emad)	Profile, matching	Returns a fixed sample JSON, even before the real API works
Auth logic (Yousef)	Requests and FCM (Donia)	User document with uid and FCM token field created at first sign-in
AI matching v1 (Emad)	Wire AI feed (Yousef), prompt eval (Emad)	Input and output JSON shape frozen
Feed card widget (Bishay)	Like/skip UI (Saga), same week	Card widget merged mid-week
Requests and FCM (Donia)	Request UI (Bishay), security rules and task alerts (Saga)	Request document shape agreed and send function working
Like/skip storage (Saga)	Prompt eval (Emad)	Feedback documents stored in Firestore
Task model and rules (Donia)	Board logic (Yousef), board view (Bishay), alerts (Saga)	Collection shape and rules merged on day 1 of W5
Task board state (Yousef)	Board view hook-up (Bishay), detail screens (Donia)	State API merged

12. Task assignment by feature

Feature or task	Lead	Support	Reviewer	Due
Repo, branches, Firebase project, data model	Donia	Yousef	Emad	22 Oct
Design system, wireframes and screens	Bishay	Yousef	Donia	29 Oct
Flutter project, flavors, navigation	Yousef	Saga	Donia	22 Oct
Auth logic, welcome and home	Yousef (logic), Donia (UI)	Bishay	Saga	29 Oct
Profile and GitHub data	Emad (data), Donia (screens)	Yousef	Bishay	29 Oct
AI matching and match feed	Emad (matching), Bishay (feed)	Donia, Yousef	Saga	29 Oct (v1), 12 Nov (tuned)
Like/skip feedback	Saga	Bishay	Emad	29 Oct
Collaboration requests and FCM	Donia	Bishay, Yousef	Saga	5 Nov
Animations and request screens	Bishay	Yousef	Donia	5 Nov

Feature or task	Lead	Support	Reviewer	Due
Security rules, permissions, first obfuscated build	Saga	Yousef	Donia	12 Nov
Task board (if time allows)	Donia (model, detail), Yousef (logic), Bishay (view), Saga (alerts)	All	Rotates	26 Nov
Free and Pro release builds, install link	Yousef	Saga	Donia	30 Nov
Documentation and README	Saga	All	Bishay	30 Nov
Final presentation and demo	Bishay	All	Donia	30 Nov

> Reviewer names are proposed so that nobody reviews their own work and each person reviews a different layer. Adjust them in Jira during sprint planning.

PART 4

Risk Assessment and Mitigation Plan

13. Risks and mitigation

Risk	Likelihood	Impact	Mitigation	Owner
Fixed deadline (30 Nov)	Medium	High	Must-have list, core freeze on 12 Nov, task management only if the core is done	Donia
Poor match quality	Medium	High	Prompt evaluation set and like/skip feedback loop	Emad
GitHub API rate limits	Medium	Medium	Cache profile data and fetch in batches	Emad
AI model cost or latency	Medium	Medium	Cache results and limit calls per user	Emad
LLM API key exposed in the app	Low	High	Call the model from a Cloud Function; keep the key as a function secret, never in the repo	Yousef
Team member unavailable	Medium	Medium	Pair work, code reviews and shared knowledge. The rotating model reduces single points of failure.	Donia
Rotation slows people down at first	Medium	Medium	Pair on day one of each slice; keep W1 as pairs; document data shapes before building	Donia
Scope creep	Medium	Medium	Task management starts only after the core freeze	Donia
Obfuscation breaks release build	Low	Medium	Test an obfuscated build by 12 Nov, not the last day	Saga
Install link is hard to use (sideloading)	Low	Medium	Write a 5-line install guide in the README; test the link on a clean device	Yousef
Merge conflicts	High	Low	Small branches and frequent pull requests	Yousef
Firebase misconfiguration	Low	High	Security rules review and separate dev and prod projects	Donia
Unexpected Firebase costs	Low	Medium	Budget alerts, stay within free quota, confirm plan before Week 3	Donia

14. Quality assurance and security controls

Testing

- Unit tests for matching logic and data models.
- Widget tests for key screens.
- Integration tests for sign-in, match feed and collaboration requests.
- Prompt evaluation set to compare matching quality across prompt versions.
- Manual testing on at least two real Android devices and one emulator.
- Peer testing: each feature is tested by someone other than its author before merge.

Security (Android software stack security model)

- **Runtime permissions:** request only what is needed (for example notifications), at the moment of use, with a clear explanation.
- **Authentication:** Firebase UI Auth with secure token handling.
- **Data access:** Firestore security rules so users read and write only what they should.
- **Secrets:** no API keys or tokens in the repository; the LLM key lives in a Cloud Function secret.
- **Code protection:** obfuscation on release builds, with symbol files stored safely.
- **Network:** all traffic over HTTPS.
- **Review:** security checklist completed before release.

PART 5

KPIs (Key Performance Indicators)

15. Metrics for project success

Targets are proposed and can be adjusted with your supervisor. Technical KPIs are measured during testing and the pilot, using Firebase Crashlytics, Firebase Performance Monitoring and app logs.

KPI	Target	How it is measured	Owner
Match response time	Match results shown in 5 seconds or less for 95% of requests	Timing logs and Firebase Performance Monitoring	Emad
App start time	Cold start in 3 seconds or less on the test devices	Manual timing and Performance Monitoring	Yousef
Notification delivery	FCM alert received within 10 seconds in 95% of tests	Test script with timestamps	Donia
System uptime	99.5% or higher for backend services during testing and the pilot	Firebase status and monitoring	Donia
Stability	99% or more crash-free sessions	Firebase Crashlytics	Saga
Test quality	All tests pass on main; 70% or more coverage of matching logic; 0 critical bugs open at release	CI results and Jira bug list	Saga
Security	100% of the security checklist completed	Checklist review before release	Saga
Match quality	At least 50% of top-5 matches rated relevant by pilot users	Like and skip feedback	Emad
Prompt improvement	Relevance score improves by 15% or more from first to final prompt version	Evaluation set results	Emad
User adoption rate	At least 20 pilot users sign up and 60% complete their profile	Firebase Auth and Firestore counts	Donia
Collaboration activity	At least 10 collaboration requests sent during the pilot	Firestore request records	Donia
Sprint delivery	80% or more of sprint tasks completed each sprint	Jira sprint reports	Donia
Milestones	All 4 milestones met on time; 100% of must-have scope delivered	Milestone review and Jira	Donia
Team coverage (new)	Every member owns at least one slice in both UI and backend by 12 Nov	Jira slice owners	Donia

16. Course requirements checklist

Requirement	Lead	Due
Git and GitHub with branching and pull requests	Donia	From 16 Oct
Firebase UI Auth (Google, email)	Yousef	By 29 Oct
Free and Pro flavors	Yousef	By 22 Oct, builds by 30 Nov
FCM notifications	Donia	By 5 Nov
Implicit and advanced animations	Bishay	By 5 Nov
Security best practices and runtime permissions	Saga	By 12 Nov
Code obfuscation and Android App Bundle	Yousef, Saga	By 30 Nov
Technical documentation and README	Saga, all	By 30 Nov

PART 6

Future Work and Ideas

17. Roadmap after 30 Nov and ideas to make DevSync stronger

These items are outside the 30 Nov scope. They show where the project can grow and give the supervisor a clear picture of the vision.

Phase	Idea	Why it strengthens DevSync
Next (Dec 2026)	Run the pilot with 20+ users and collect like/skip data	Gives real evidence for the match quality and adoption KPIs
Next	Automated prompt evaluation in CI	Every prompt change is checked against the evaluation set before merge
Next	Firebase Remote Config for prompt versions, with A/B tests	Switch and compare prompt versions without a new release
Next	Firebase App Check and basic rate limiting	Protects the backend and the LLM budget from abuse
Later	Web platform on the same Firebase backend	Reaches maintainers who work on desktop
Later	iOS build	Wider audience; Flutter code is mostly reusable
Later	Real Pro plan with in-app payments	Turns the Pro flavor into a business model
Later	Issue-level matching (good first issues) and skill-gap suggestions	Moves from 'which project' to 'which task, and what to learn first'
Later	Maintainer search for contributors, in-app chat and endorsements	Completes the two-sided marketplace and builds trust